Silicon-Aware Local RTRL on a Hardware-Primitive Grid Lattice

Thomas Pluck

April 23, 2026

Abstract

A recurrent architecture built from one analog-crossbar primitive (Gm–C integrator + tanh + learnable bias) replicated over a rectangular $N \times M$ grid. Each cell has two weighted inputs (from-left and from-above) and one output routed to both its right and below neighbours: the left shoreline takes external input, the bottom shoreline produces block output, and the top/right shorelines are nulled. Four learnable scalars per cell. The encoder (left shoreline) and decoder (bottom shoreline) carry no learned parameters. We train the grid on row-wise sequential MNIST with a local SnAP-1 rule (two descendant cross-traces per cell); per-parameter grid gradients match BPTT at float-64 ϵ on a single forward step, confirming numerical correctness. Training numbers are in Section 4.

1 Introduction and contribution

Setting. Analog neuromorphic crossbars can implement a Gm–C low-pass integrator with a tanh-shaped output stage cheaply and uniformly. The design question is not whether such a substrate can be made to learn at all, but what *architecture* and *rule* fit inside its constraints: no backpropagation-through-time (no global backward pass), no per-parameter adaptive optimiser state, bounded per-cell analog state, finite arithmetic precision, fixed input wiring, and spatial locality of the learning signal. We take those constraints as the scope of the problem rather than axes to apologise along.

Contributions.

1. **A grid primitive with two-in/two-out connectivity.** An $N \times M$ rectangular grid of cells where each cell receives inputs from its left and above neighbours and drives output to its right and below neighbours. Left shoreline is external input, bottom shoreline is block output, remaining shorelines are nulled. Four learnable scalars per cell $(\log \tau, w_{\mathrm{L}}, w_{\mathrm{T}}, b)$.

2. **A local rule with per-descendant cross-traces.** SnAP-1 on this topology tracks one self-trace per learnable parameter plus one cross-trace per descendant (2 per cell). Single-step gradients match BPTT at float-64 ϵ. Full-sequence gradients diverge with T because each cell has two descendant paths that SnAP-1 only partially captures—a harder truncation regime than the tree variant, which had one descendant per cell.

3. **A silicon-aware characterisation.** Within the scope the hardware imposes, we map six design-relevant axes: weight/state/trace precision, SnAP-k accuracy vs. state budget, timescale range, topology (T, K, B, s) at matched cell count, capacity scaling, and transfer to an RL task (MinAtar) and a neuromorphic benchmark (SHD). The output is a design-space picture, not a leaderboard.

Scope. The left and bottom shorelines of the grid are used as input and output respectively, carrying no learned parameters; when input dimension matches grid height the encoder is pure identity wiring. The only trainable parameters are the four per-cell scalars.

Relation to prior work. The RTRL taxonomy has been mapped by Marschall, Cho and Savin [2]; sparse and stochastic approximations include UORO [3], KF-RTRL [4], and SnAP-k [5]. RFLO [6] and e-prop [7] are the closest local-rule neighbours. OSTL [8] and Zucchet *et al.* [10] both exploit structural sparsity for exact online learning in specific architecture classes; our tree variant is a generalisation. On the *dendritic* side, models combining multi-compartment neurons with local learning form a growing literature. The nearest work is Yin *et al.* [15]: they apply e-prop (equivalent to SnAP-1) to a two-compartment spiking dendritic neuron, on SHD. Differences from the present work: (i) we generalise from SnAP-1 to arbitrary k on an ancestor chain and make the structural-sparsity argument explicit; (ii) we work with arbitrary tree depth and branching (T, K, B, s) rather than a fixed two-compartment topology; (iii) the Gm–C+ tanh primitive is fully differentiable, whereas LIF-based work requires surrogate gradients—a genuine simplification for an exact-RTRL argument and for analog mapping. Zhang *et al.*'s TC-LIF [17], Huang *et al.*'s DendSN [18], and Feng *et al.*'s TS-LIF [19] are parallel dual-compartment proposals. Biologically motivated multi-compartment learning rules include Sacramento *et al.* [12], Guerguiev *et al.* [13], and Payeur *et al.* [14]. On the ML-side dendrite literature: Beniaguev *et al.* [20] show single cortical neurons match small deep ANNs; Iyer *et al.* [21] use active dendrites for multi-task generalisation; Poirazi and Papoutsi [22] review the field. On the analog hardware side, Cartiglia *et al.* [23] and related work from the Indiveri group including Maryada *et al.* [24] prototype dendritic learning in silicon; the review of Acharya *et al.* [25] covers the broader landscape.

What is novel here is the combination of: (i) a rectangular-grid primitive with 2-in/2-out connectivity and native shoreline I/O, giving the same filter+tanh+bias cell a very different sensitivity structure from the tree variant [9]; (ii) a SnAP-1 specialisation that tracks one cross-trace per immediate descendant (2 per cell) rather than a single ancestor chain; (iii) empirical quantification of the multi-hop-descendant truncation cost that SnAP-1 pays on this topology, which is significantly higher than on the tree. What is *not* novel is the SnAP-k framework, the per-group-LR optimiser story, or the fixed-shoreline-encoder approach.

Code. https://github.com/ThomasPluck/grid-neurons

2 Architecture

2.1 The canonical cell

Every grid cell is the same first-order integrator followed by tanh. Cell (i, j) at position row i, column j receives two inputs: one from its left neighbour $y_{i,j-1}$ and one from its top neighbour $y_{i-1,j}$, each weighted independently:

$$\mathrm{drive}_{ij}(t) = w^{\mathrm{L}}_{ij}\, y_{i,j-1}(t) + w^{\mathrm{T}}_{ij}\, y_{i-1,j}(t), \tag{1}$$

$$\tau_{ij}\frac{\mathrm{d}s_{ij}}{\mathrm{d}t} = -s_{ij} + \mathrm{drive}_{ij}(t), \tag{2}$$

$$y_{ij}(t) = \tanh\big(s_{ij}(t) + b_{ij}\big). \tag{3}$$

Zero-order-hold discretisation:

$$a_{ij} = \mathrm{e}^{-\Delta t/\tau_{ij}}, \quad c_{ij} = 1 - a_{ij}, \qquad s_{ij}(t{+}1) = a_{ij}\, s_{ij}(t) + c_{ij} \cdot \mathrm{drive}_{ij}(t{+}1). \tag{4}$$

Each cell carries four learnable scalars, $\theta_{ij} = \{\log \tau_{ij}, w_{ij}^{\mathrm{L}}, w_{ij}^{\mathrm{T}}, b_{ij}\}$. Output $y_{ij} \in (-1, 1)$ is bounded, so no normalisation is required.

2.2 Topology: rectangular grid with shoreline I/O

A grid block is a rectangle of cells indexed (i, j) with $i \in \{0, \ldots, N-1\}$, $j \in \{0, \ldots, M-1\}$. Connectivity is fixed and feed-forward within a timestep:

- Each cell has **two inputs** from its left and top neighbours, and **two output edges** routing y_{ij} to its right and below neighbours.
- **Left shoreline** (j=0): external input. Each cell in column 0 receives one external channel in place of a left neighbour.
- **Bottom shoreline** (i=N−1): block output. Each cell in row $N-1$ is one of the M output values.
- **Top shoreline** (i=0): the missing top neighbour is treated as zero. **Right shoreline** (j=M−1): the outgoing right-edge is dropped.

Forward traversal is raster order (left-to-right within a row, then top-to-bottom): each cell's inputs come from cells already updated this step. Backward traversal is the reverse. The grid has NM cells and $\sim 2NM$ internal edges, so the forward pass is $O(NM)$ per timestep.

2.3 Encoder and decoder: native I/O

A block is one grid; it holds every trainable parameter. Encoder and decoder are defined by the grid's shoreline geometry and carry no learned parameters.

Encoder. The left shoreline directly accepts external input: cell $(i, 0)$'s left-input at each timestep is external channel $x_i(t)$. For tasks where $N_{\mathrm{ext}} = N$ (input dim = grid height) this is literally an identity wiring; when $N_{\mathrm{ext}} \neq N$ an optional fixed sparse-random projection $W_{\mathrm{in}} \in \mathbb{R}^{N \times N_{\mathrm{ext}}}$ maps external channels to left-shoreline drives. The projection is drawn once at initialisation and not trained.

Decoder. The bottom shoreline is the M-dimensional block output. Cell $(N-1, j)$'s output $r_j(t) = y_{N-1,j}(t) \in (-1, 1)$ is used directly. For classification we sum over time, $\hat{\ell}_j = \sum_t r_j(t)$, and feed $\hat{\ell}$ to a standard softmax–cross-entropy loss. No learned readout weights.

Per task.

- *Row-wise MNIST*: N=28 (one grid row per input channel), M=10 (one bottom-shoreline cell per class). $N_{\mathrm{ext}} = N$, identity encoder. Decoder is summed-readout softmax-CE.

In every experiment the only trainable parameters are the per-cell $(\log \tau_{ij}, w_{ij}^{\mathrm{L}}, w_{ij}^{\mathrm{T}}, b_{ij})$ quadruples. The grid carries the whole representation.

3 Sparse grid-RTRL via SnAP-1

3.1 Structural sparsity of the sensitivity tensor

Parameter $\theta_{ij} \in \{\log \tau_{ij}, w^{\mathrm{L}}_{ij}, w^{\mathrm{T}}_{ij}, b_{ij}\}$ appears only in cell (i, j)'s update, and its influence spreads only to downstream cells on the grid:

$$\frac{\partial s_{k\ell}(t)}{\partial \theta_{ij}} = 0 \quad \text{unless } k \geq i \text{ and } \ell \geq j. \tag{5}$$

Unlike the one-in/one-out tree (ancestor chain of length $O(\log N)$), the grid's two-in/two-out structure makes the downstream cone grow quadratically with distance: at Manhattan distance d from (i, j) there are $d{+}1$ descendants, and the set within distance k has size $\binom{k+2}{2}$. A full grid-RTRL would carry $O(NM)$ traces per parameter and store $\sim(NM)^2$ total.

3.2 SnAP-1 traces

We track self-traces for the three filter-sensitive parameters and *one cross-trace per immediate descendant* (below and right):

$$e^{\mathrm{L}}_{ij}(t) \equiv \partial s_{ij}/\partial w^{\mathrm{L}}_{ij}, \qquad e^{\mathrm{T}}_{ij}(t) \equiv \partial s_{ij}/\partial w^{\mathrm{T}}_{ij}, \qquad \eta_{ij}(t) \equiv \partial s_{ij}/\partial \log \tau_{ij}, \tag{6}$$

$$\epsilon^{(1,d)}_{ij,\theta}(t) \equiv \partial s_d(t)/\partial \theta_{ij}, \qquad d \in \{\text{below}, \text{right}\}. \tag{7}$$

Total per-cell state at SnAP-1: 3 self-traces + 2 descendants $\times$ 4 parameter types = 11 scalars, a constant factor more than the tree primitive. Cross-trace update, for each θ and each descendant d:

$$\epsilon^{(1,d)}_{ij,\theta}(t{+}1) = a_d\, \epsilon^{(1,d)}_{ij,\theta}(t) + c_d\, \kappa_d\, (1 - y^2_{ij})\, f_{\theta,ij}(t{+}1), \tag{8}$$

where κ_d is the weight that couples y_{ij} into d's drive (w^{T}_d if d is the below-neighbour, w^{L}_d if the right-neighbour) and $f_{\theta,ij}$ is the self-derivative.

3.3 Backward pass

Messages start at the bottom shoreline with $m_{N-1,j}(t) = \partial L/\partial r_j(t)$ and propagate in reverse raster order. Each cell's incoming message is the sum of contributions from its two descendants through the spatial Jacobians $\partial y_d/\partial y_{ij} = \kappa_d\,(1{-}y^2_d)\, c_d$. The per-timestep gradient combines spatial + SnAP-1 terms with the past-only (anti-double-count) subtraction:

$$\frac{\mathrm{d}L_t}{\mathrm{d}\theta_{ij}} = m_{ij}(t)\,(1{-}y^2_{ij})\, f_{\theta,ij}(t) + \sum_{d \in \{\text{below},\text{right}\}} m_d(t)\,(1{-}y^2_d)\,\big[\epsilon^{(1,d)}_{ij,\theta}(t) - c_d\,\kappa_d\, f_{\theta,ij}(t)\big]. \tag{9}$$

4 Row-wise MNIST

Full-scale row-wise MNIST: 60,000 training images, 10 epochs, batch 32, Adam $\eta{=}3{\times}10^{-3}$, $\tau \in [1, 20]$ ms, input gain 10. Grid dimensions match the task: $N{=}28$ (left-shoreline rows, one per input channel) and $M{=}10$ (bottom-shoreline columns, one per class). Identity encoder, summed-readout softmax-CE decoder, no learned encoder/decoder parameters. Local SnAP-1 rule (two descendant cross-traces per cell, eleven per-cell state scalars).

The grid trains end-to-end on full-scale MNIST under SnAP-1, reaching 0.522 final val accuracy (vs. random-chance 0.10). For reference the tree variant at a similar per-cell state budget reports $\sim$0.58 [9]; a BPTT comparison on the same grid architecture would be welcome but is $\gg 1$ hr per 10-epoch run on our CPU setup and is deferred.

Table 1: Grid (28×10) SnAP-1 on row-wise MNIST. Single seed.

epoch	train loss	train acc	val acc	cum. time
0	2.106	0.252	0.374	171 s
1	1.887	0.320	0.324	338 s
2	1.783	0.364	0.438	502 s
3	1.570	0.467	0.500	665 s
4	1.507	0.491	0.503	829 s
5	1.526	0.495	0.494	993 s
6	1.584	0.456	0.430	1155 s
7	1.554	0.474	0.479	1318 s
8	1.476	0.494	0.512	1485 s
9	1.433	0.517	**0.522**	1659 s

A Architectural ablations

No residual feed-through. An earlier version of the cell included a learnable residual-feedthrough coefficient w_{pass} providing a linear shortcut around the tanh nonlinearity. We removed it: empirical sweeps showed it offered no material improvement in either BPTT or local-rule training after LR ratios were tuned, and it made the forward-pass bound less clean (residual outside tanh) or killed its gradient-flow advantage (residual inside tanh). The final primitive in Section 2 is accordingly the minimal filter+bias+tanh with no shortcut.

References

[1] R. J. Williams and D. Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.

[2] O. Marschall, K. Cho, and C. Savin. A unified framework of online learning algorithms for training recurrent neural networks. *Journal of Machine Learning Research*, 21(135):1–34, 2020.

[3] C. Tallec and Y. Ollivier. Unbiased online recurrent optimization. In *International Conference on Learning Representations*, 2018.

[4] A. Mujika, F. Meier, and A. Steger. Approximating real-time recurrent learning with random Kronecker factors. In *Advances in Neural Information Processing Systems*, 2018.

[5] J. Menick, E. Elsen, U. Evci, S. Osindero, K. Simonyan, and A. Graves. A practical sparse approximation for real time recurrent learning. In *International Conference on Learning Representations*, 2021.

[6] J. M. Murray. Local online learning in recurrent networks with random feedback. *eLife*, 8:e43299, 2019.

[7] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature Communications*, 11:3625, 2020.

[8] T. Bohnstingl *et al.* Online spatio-temporal learning in deep neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 2023.

[9] T. Pluck. Silicon-aware local RTRL on a hardware-primitive dendritic tree. https://github.com/ThomasPluck/tree-neurons, 2026.

[10] N. Zucchet, R. Meier, S. Schug, A. Mujika, and J. Sacramento. Online learning of long-range dependencies. In *Advances in Neural Information Processing Systems*, 2023.

[11] A. Orvieto, S. L. Smith, A. Gu, A. Fernando, C. Gulcehre, R. Pascanu, and S. De. Resurrecting recurrent neural networks for long sequences. arXiv:2303.06349, 2023.

[12] J. Sacramento, R. Ponte Costa, Y. Bengio, and W. Senn. Dendritic cortical microcircuits approximate the backpropagation algorithm. In *Advances in Neural Information Processing Systems*, 2018.

[13] J. Guerguiev, T. P. Lillicrap, and B. A. Richards. Towards deep learning with segregated dendrites. *eLife*, 6:e22901, 2017.

[14] A. Payeur, J. Guerguiev, F. Zenke, B. A. Richards, and R. Naud. Burst-dependent synaptic plasticity can coordinate learning in hierarchical circuits. *Nature Neuroscience*, 24(7):1010–1019, 2021.

[15] B. Yin *et al.* Training high-performance deep learning networks with two-compartment dendritic neurons using e-prop. arXiv:2402.15969, 2024.

[16] K. Young and T. Tian. MinAtar: An Atari-inspired testbed for thorough and reproducible reinforcement learning experiments. arXiv:1903.03176, 2019.

[17] S. Zhang *et al.* TC-LIF: A two-compartment spiking neuron model for long-term sequential modelling. arXiv:2308.13250, 2024.

[18] X. Huang *et al.* DendSN: A dendritic spiking neuron model for deep neural networks. arXiv:2412.06355, 2024.

[19] L. Feng *et al.* TS-LIF: A temporal-separation two-compartment spiking neuron. In *International Conference on Learning Representations*, 2025.

[20] D. Beniaguev, I. Segev, and M. London. Single cortical neurons as deep artificial neural networks. *Neuron*, 109(17):2727–2739, 2021.

[21] A. Iyer *et al.* Avoiding catastrophe: Active dendrites enable multi-task learning in dynamic environments. *Neural Networks*, 155:1–16, 2022.

[22] P. Poirazi and A. Papoutsi. Illuminating dendritic function with computational models. *Nature Reviews Neuroscience*, 21(6):303–321, 2020.

[23] M. Cartiglia, A. Rubino, S. Narayanan, C. Frenkel, G. Haessig, G. Indiveri, and M. Payvand. Stochastic dendrites enable online learning in mixed-signal neuromorphic processing systems. arXiv:2201.10409, 2022.

[24] Maryada *et al.* Canonical cortical microcircuit with analog dendritic learning on neuromorphic hardware. bioRxiv 2025.03.28, 2025.

[25] J. Acharya *et al.* Dendritic computing: Branching deeper into machine learning. *Neuroscience*, 489:275–289, 2021.